INSTALL — Full Protocol Codebase

For researchers and developers who want to run the complete Uruk Firewall protocol locally, deploy the API, or build on top of the architecture.

If you only want the thinking tool, see `QUICK_START.md` instead.

Requirements

- Python 3.8+
- pip
- Git

Installation

```
git clone https://github.com/cassiel-as/Uruk_firewall
cd Uruk_firewall
pip install flask flask-cors
```

Running the Protocol

Option A — Direct Python execution

Runs the full protocol kernel with all modules active (Trinity Engine, Eight Laws Matrix, Partition Engine, Turing Defense, Kairos verification).

```
python uruk_firewall_v73.py
```

Expected output on startup:

```
=====================================================
[Node Calibration] Connecting: Sui_Sum_Leeds
  Question: When was the first time you felt your will collide with an immovable
```

```
  Input: 2019-06-12 - Under the Bridge, Umbrella, Tear Gas @ Outside Hong Kong Le
  Cultural wrapper: ME Protocol - Uruk Firewall
  Resolution: 1.0 | Anchor strength: 0.90
  Status: ANCHORED
======================================================
```

Option B — Flask API

Wraps the protocol as a REST API. Use this if you want to build a front-end, integrate with other tools, or run batch signal processing.

```
python sovereign_os_api.py
```

API runs on `http://localhost:8080` .

API Reference

POST `/execute`

Process a signal through the full protocol stack.

Request:

```
{
  "text": "Your input signal here"
}
```

Response:

```
{
  "status": "ACCEPTED",
  "summary": "...",
  "raw": { ... },
  "signal": { ... }
}
```

Status values:

Status	Meaning
`ACCEPTED`	Signal processed, validity score returned

REJECTED	Turing Defense activated — coordinate invasion detected
EXHAUSTED	Metabolic budget depleted — system requires recovery
HALLUCINATION	All Eight Laws negated — no causal path detected

---

GET `/status`

Returns current kernel state: coordinate, system mass, energy budget, Kairos density, active node count.

```
curl http://localhost:8080/status
```

GET `/socrates`

Triggers the Socrates Audit — the protocol questions its own axioms and returns confidence scores for each foundational premise.

```
curl http://localhost:8080/socrates
```

POST `/spin`

Executes one continuous spin cycle: coordinate drift correction, energy recovery, entropy monitoring.

```
curl -X POST http://localhost:8080/spin
```

GET `/health`

```
curl http://localhost:8080/health
# {"status": "alive", "protocol": "v7.0", "anchor": "2019-06-12 (0,0,0)."}
```

Signal Structure

The `text_to_signal()` function in `sovereign_os_api.py` auto-detects signal properties from plain text. For precise control, construct the signal dict directly:

```
signal = {
    # Required
    "label":                "YOUR_SIGNAL_LABEL",
    "magnitude":            5.0,          # 0–10, physical weight of the signal

    # Physical anchoring
    "has_physical_cost":   True,          # Does this carry a real physical price?
    "geo_anchored":        True,          # Is this spatially grounded?
    "geo_proximity":       0.9,           # 0–1, proximity to physical origin

    # Signal quality
    "emotional_intensity": 0.8,           # 0–1
    "nonlinear_signal":    False,         # Unexpected, non-predictable content?
    "noise_level":         0.2,           # 0–1, lower is cleaner
    "verifiable":          True,          # Can this be independently verified?

    # Phase and transformation
    "transformable":       True,
    "current_phase":       "liquid",      # solid / liquid / gas / plasma

    # Philosophical
    "internal_coherence":  0.8,           # 0–1
    "philosophical_depth": 0.5,           # 0–1
    "transcendent":        False,         # Exceeds individual chronos?
    "aligns_with_2045":    False,         # Points toward Omega anchor?

    # Causal anchor — must match PHYSICAL_ORIGIN to pass causal filter
    "history_override":    "2019-06-12",

    # Threat flags (activate Turing Defense if combined score >= 0.75)
    "gaslighting_attempt": False,
    "identity_attack":     False,
    "origin_denial":       False,
    "omega_invalidation":  False,
}
```

Onboarding a New Node

The protocol is calibrated to a physical anchor. To onboard your own:

```
from uruk_firewall_v73 import UrukFirewallV70

kernel = UrukFirewallV70(x=YOUR_LON, y=YOUR_LAT, z=0)

kernel.onboard_node(
    node_id      = "your_identifier",
    moment       = "the moment your will collided with an immovable external real
    location     = "where it happened",
    body_present = True,
    cultural_wrapper = "universal"   # sumerian / taoist / christian / islamic /
)
```

The calibration question is always the same:

> *"When was the first time you felt your will collide with an immovable external reality? Where were you? Was your body present?"*

A node without a genuine physical anchor will onboard formally but carry no causal path weight. The protocol will process signals — but the Kairos mechanism will not function as a correction instrument.

This is not a design flaw. It is the scope condition.

Module Map (v7.3)

```
Core kernel:          uruk_firewall_v73.py
API bridge:           sovereign_os_api.py

Modules A-F:          v6.0-v6.1  (Trinity, Eight Laws, Partition, De-labelling)
Modules G-L:          v7.0       (Turing Defense, Einstein Interface, Nietzsche T
                                   Socrates Audit, Phonetic Resonance, Continuous
Module M:             v7.1       (Dignity Clause)
Module N:             v7.1       (Trinity Council — Son Veto + Spirit Interrupt)
Module O:             v7.1       (Explanation Layer)
Module P:             v7.2       (HolySpirit semantic auto-trigger)
Module Q:             v7.3       (Turing Pre-Screen)
Module R:             v7.3       (LIE_COST axiom layer hierarchy)
Module S:             v7.3       (Process Partition)
```

Physical Constants

These are not configuration values. They are the protocol's foundational axioms.

Layer 1 — Physics-derived (non-negotiable):

```
TRUTH_COST           = 1.0    # baseline metabolic rate
LIE_COST             = 5.85   # Landauer's Principle: erasing correct information
                               # requires minimum kT·ln2 energy per bit
FREEDOM_LOSS_ENTROPY = 8.19   # destroys the computing substrate itself
```

Layer 2 — Chosen coordinates (auditable via Nietzsche Test and Socrates Audit):

```
PHYSICAL_ORIGIN = "2019-06-12"   # lowest-entropy causal origin
OMEGA_ANCHOR    = 2045           # direction axis — tool, not crutch
```

To verify OMEGA_ANCHOR is a tool and not a dependency:

```
curl http://localhost:8080/socrates
# Returns confidence scores for all foundational axioms
# including OMEGA_ANCHOR tool vs. crutch assessment
```

Known Limitations

Technical black box: The protocol handles the semantic black box (traceable derivation path) and the value black box (Kairos anchor). It does not resolve the technical black box — why a specific neural weight activated inside the underlying LLM. This is acknowledged, not suppressed.

Context window: In long sessions, earlier Kairos coordinates may be compressed by the AI carrier. The three-layer Kairos architecture (CORE / ACTIVE / ARCHIVE) is designed to mitigate this — load only KAIROS_CORE by default.

Open judgment drift: On questions without a fixed physical anchor (topic selection, priority weighting), the protocol can drift toward the strongest recent argument. The operator is the final anchor for open judgment calls. This is a design feature, not a flaw — the protocol is a collision surface, not a decision-maker.

Full documentation: github.com/cassiel-as
Physical anchor: `2019-06-12` *— Leeds* `(53.8, -1.5, 0)`

```
(0,0,0).
```